



UNIVERSITÀ DEGLI STUDI DI MILANO

Sicurezza & Privacy

Laboratorio 2: Assembly

Andrea Monzani

Prof. Danilo Bruschi



Human-Readable Machine Language

- Computers like ones and zeros... **0001110010000110**
- Humans like symbols... **MOV rax, rdx ; rdx → rax**
- **Assembly** is a programming language in which instructions can be directly converted into machine instructions.
 - ISA-specific: close correspondence between symbols and instruction set
 - mnemonics for opcodes
 - labels for memory locations
 - additional operations for allocating storage and initializing data
- **Assembler** is the program which convert and assembly program into an executable one



Why use it

- Assembly is often used for performance-critical parts of a program, although it is difficult to outperform a good C++ compiler for most programmers.
- Assembly knowledge is useful for debugging code – sometimes a compiler makes incorrect assembly code and stepping through the code in a debugger helps locate the cause. Code optimizers sometimes make mistakes.



Why use it

- Another use for assembly is interfacing with or fixing code for which you have no source code.
- Disassembly lets you change/fix existing executables.
- Assembly is necessary if you want to know how your language of choice works under the hood – why some things are slow and others are fast.
- Finally, assembly code knowledge is **indispensable when diagnosing malware.**



Historic notes

- For years, PC programmers used x86 assembly to write performance-critical code. However, 32-bit PCs are being replaced with 64-bit ones, and the underlying assembly code has changed
- x64 is a generic name for the 64-bit extensions to Intel's and AMD's 32-bit x86 instruction set architecture (ISA). AMD introduced the first version of x64, initially called x86-64 and later renamed AMD64. Intel named their implementation IA-32e and then EM64T, they are a little different, their intersection is called **x64**

Too many Assemblers

- There are many x64-capable assemblers such as the **Netwide Assembler NASM** , a **NASM** rewrite called **YASM**, the fast **Flat Assembler FASM** , and the traditional Microsoft **MASM**.
- Each assembler has varying support for other assemblers' macros and syntax, but assembly code is not source compatible across assemblers
- **We will use GCC/AS**

Assembly Language Example



[Simple Calc Example](#)

Assembly Language Syntax

- Each line of an assembly program is one of the following:
 - an instruction
 - an assembler directive (or pseudo-op)
 - a comment
- Whitespace (between symbols) and case are ignored.
- Comments (beginning with “#”) are also ignored.
- An instruction has the following format:

LABEL: OPCODE OPERANDS COMMENTS

The diagram illustrates the syntax of an assembly instruction: **LABEL: OPCODE OPERANDS COMMENTS**. Annotations include: a green arrow pointing to **LABEL:** with the word *optional* in green below it; a red arrow pointing to **OPCODE** with the word *mandatory* in red below it; and a green arrow pointing to **OPERANDS** with the number **8** in green below it. A horizontal green line spans the width of the instruction format.

Assembly Flavors

INTEL

```
sum:
    lea    eax, [rdi+rsi]
    ret

.LC0:
    .string "%d"
.LC1:
    .string "Sum of %d and %d is %d\n"
main:
    sub    rsp, 24
    mov    edi, OFFSET FLAT: .LC0
    xor    eax, eax
    lea    rsi, [rsp+8]
    mov    DWORD PTR [rsp+8], 0
    call   __isoc99_scanf
```

AT&T

```
sum:
    leal    (%rdi,%rsi), %eax
    ret

.LC0:
    .string "%d"
.LC1:
    .string "Sum of %d and %d is %d\n"
main:
    subq    $24, %rsp
    movl    $.LC0, %edi
    xorl    %eax, %eax
    leaq    8(%rsp), %rsi
    movl    $0, 8(%rsp)
    call    __isoc99_scanf
```

- Different styles for representing the same assembly program
- **We will use INTEL syntax/flavor**

Assembly flavors

Prefixes.

Intex Syntax	AT&T Syntax
mov eax, 1	movl \$1, %eax
mov ebx, 0ffh	movl \$0xff, %ebx
int 80h	int \$0x80

Direction of Operands.

Intex Syntax	AT&T Syntax
instr dest, source	instr source, dest
mov eax, [ecx]	movl (%ecx), %eax

Memory Operands.

Intex Syntax	AT&T Syntax
mov eax, [ebx]	movl (%ebx), %eax
mov eax, [ebx+3]	movl 3(%ebx), %eax

Intel Syntax	AT&T Syntax
instr foo, segreg: [base+index*scale+disp]	instr %segreg: disp (base, index, scale), foo
mov eax, [ebx+20h]	movl 0x20(%ebx), %eax
add eax, [ebx+ecx*2h]	addl (%ebx, %ecx, 0x2), %eax
lea eax, [ebx+ecx]	leal (%ebx, %ecx), %eax
sub eax, [ebx+ecx*4h-20h]	subl -0x20(%ebx, %ecx, 0x4), %eax

<https://imada.sdu.dk/u/kslarsen/dm546/Material/IntelInATT.htm>

Labels and Comments

- **Label**

- placed at the beginning of the line
- assigns a symbolic name to the address corresponding to line

- ex:

```
LOOP: add rax, 3
      jmp LOOP
```

- **Comment**

- anything after a predefined symbol (sharp) is a comment
- ignored by assembler
- used by humans to document/understand programs
- tips for useful comments:
 - avoid restating the obvious, as “decrement R8”
 - provide additional insight, as in “accumulate product in R9”
 - use comments to separate pieces of program

Operands

- Operands contain data on which perform the computation:
 - Immediate values
 - Registers
 - Memory
- Each instruction can contain one or more operands:
 - The first operand (if present) is the “destination”
 - The second operand (if present) is the “source” of the operation
 - Either the destination or the source can be a memory operand (if the instruction allows it)
 - [ADD documentation](#) example

Immediate Operands

- An immediate operand is a constant value directly encoded in the instruction
- If you want to load the number 128 (immediate operand) into **rax**:

```
mov rax, 128
```

```
mov rax, 0x80
```

- To store 0 (immediate operand) in a fixed memory location (32bit immediate addressing):

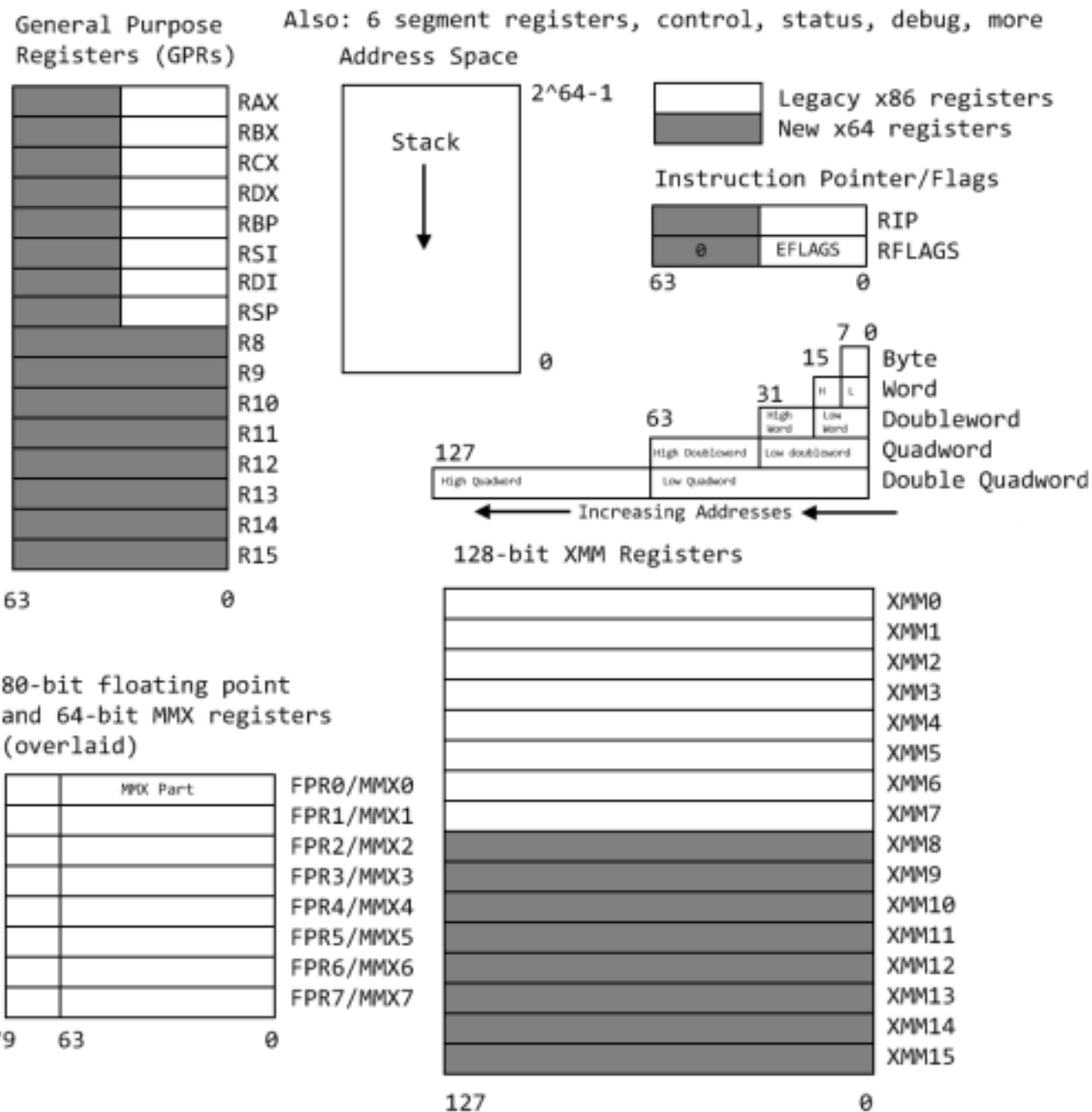
```
mov [0x01234567], 0
```

Registers operands

- Registers are operands directly accessible by the processor.
- Processors rely heavily on registers for their operations, so most of the time, data is transferred between registers and memory.
- We only care about the integer registers and, in case, the flag register. There are sixteen 64-bits integer registers:
 - RAX RCX RDX RBX RSP RBP RSI RDI
 - R8 R9 R10 R11 R12 R13 R14 R15

Registers operands

- Since the 64-bit registers allow access for many sizes and locations, we define a **byte** as 8 bits, a **word** as 16 bits, a **double word** as 32 bits, a **quadword** as 64 bits, and a double quadword as 128 bits.

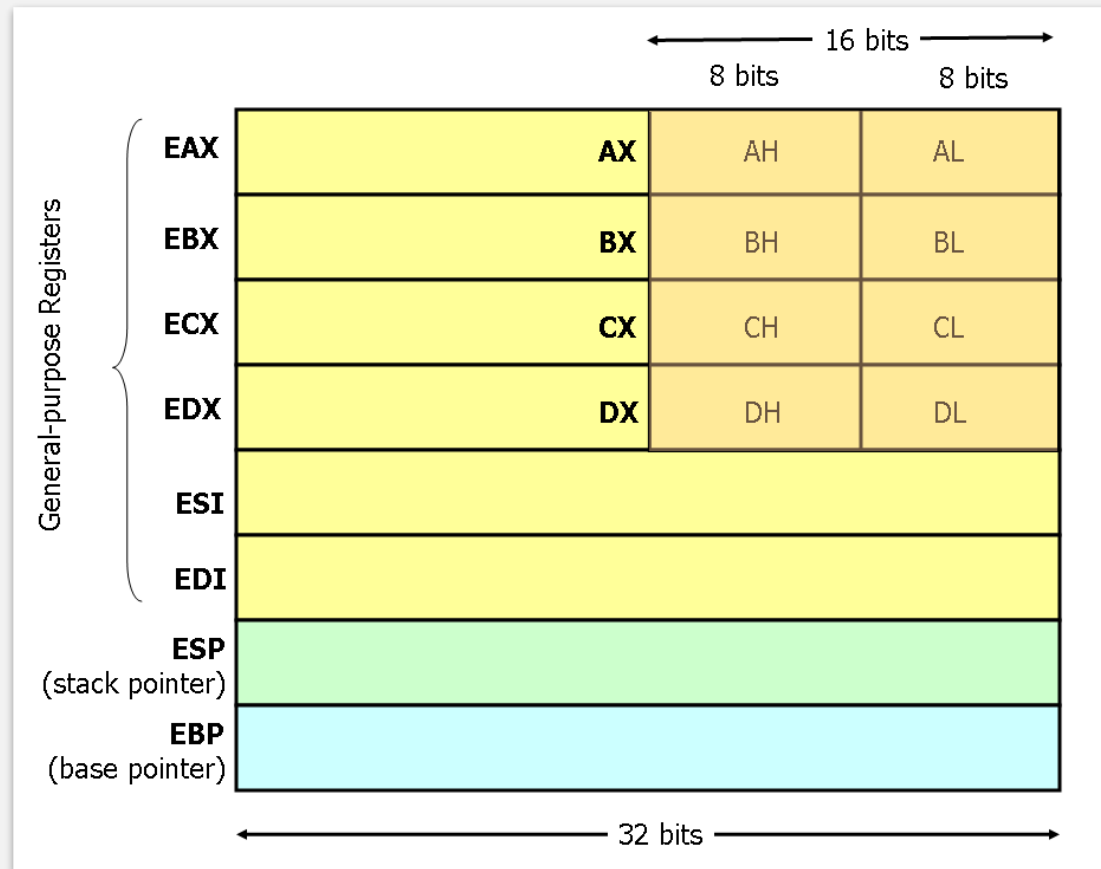


Registers operands

- There are sixteen general purpose 64-bit registers, the first eight of which are labeled (for historical reasons) RAX, RBX, RCX, RDX, RBP, RSI, RDI, and RSP.
- The second eight are named R8-R15
- By replacing the initial R with an E on the first eight registers, it is possible to access the lower 32 bits (EAX for RAX). Similarly, for RAX, RBX, RCX, and RDX, access to the lower 16 bits is possible by removing the initial R (AX for RAX), and the lower byte of the these by switching the X for L (AL for AX), and the higher byte of the low 16 bits using an H (AH for AX).
- The new registers R8 to R15 can be accessed in a similar manner like this: R8 (qword), R8D (lower dword), R8W (lowest word), R8B (lowest byte MASM style, Intel style R8L). Note there is no R8H.

Registers

64-bit register	Lower 32 bits	Lower 16 bits	Lower 8 bits
rax	eax	ax	al
rbx	ebx	bx	bl
rcx	ecx	cx	cl
rdx	edx	dx	dl
rsi	esi	si	sil
rdi	edi	di	dil
rbp	ebp	bp	bpl
rsp	esp	sp	spl
r8	r8d	r8w	r8b
r9	r9d	r9w	r9b
r10	r10d	r10w	r10b
r11	r11d	r11w	r11b
r12	r12d	r12w	r12b



Memory operands

- Memory operands are used to describe a specific memory location
 - Constant memory location (**immediate addressing**)
`mov [0x01234567], 0`
 - Register-based memory location (**register (indirect) addressing**)
`mov [rax], 0`
- Register addressing exists for different reasons
 - Not all memory addresses are known “a priori”
 - Instruction encoding limits the size of immediate addresses (64bit addresses cannot be directly encoded for example)
 - A combination of registers can be used to calculate a memory address

Immediate addressing

- These can be written in many ways. **GCC supports only plain decimal notation, “x” hex notation and binary notation**
- Remember the [] to indicate a memory access

```
200      ; decimal
0200     ; still decimal - the leading 0 does not make it octal
0200d    ; explicitly decimal - d suffix
0d200    ; also decimal - 0d prefix
0c8h     ; hex - h suffix, but leading 0 is required because c8h looks like a var
0xc8     ; hex - the classic 0x prefix
0hc8     ; hex - for some reason NASM likes 0h
310q     ; octal - q suffix
0q310    ; octal - 0q prefix
11001000b ; binary - b suffix
0b1100_1000 ; binary - 0b prefix, and by the way, underscores are allowed
```

Register addressing

- These are the basic forms of addressing:
 - $[\text{base_reg}]$
 - $[\text{base_reg} + \text{index_reg} * \text{scale}]$ scale is 1, 2, 4, or 8 only
 - $[\text{base_reg} + \text{number}]$
 - $[\text{base_reg} + \text{index_reg} * \text{scale} + \text{number}]$

```
[rbp]           ; base register only
[rcx + rsi*4]   ; base + index * scale
[rbp + rdx]     ; scale is 1
[rbx - 8]       ; displacement is -8
[rax + rdi*8 + 500] ; all four components
```

Memory Access Size

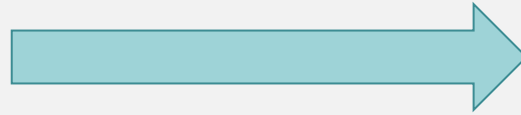
```
#include <inttypes.h>

void setzero_u8(void* buf) {
    |  *((uint8_t*)buf) = 0;
}

void setzero_u16(void* buf) {
    |  *((uint16_t*)buf) = 0;
}

void setzero_u32(void* buf) {
    |  *((uint32_t*)buf) = 0;
}

void setzero_u64(void* buf) {
    |  *((uint64_t*)buf) = 0;
}
```



```
setzero_u8:
    mov     BYTE PTR [rdi], 0
    ret

setzero_u16:
    mov     WORD PTR [rdi], 0
    ret

setzero_u32:
    mov     DWORD PTR [rdi], 0
    ret

setzero_u64:
    mov     QWORD PTR [rdi], 0
    ret
```

- GCC output available [here](#)

Memory Access Size

movzx is a necessary instruction since here gcc is optimizing the function return operation. We could also write *mov dl, BYTE PTR [rdi]*

```
uint8_t get_u8(void* buf) {  
    return *((uint8_t*)buf);  
}  
  
uint16_t get_u16(void* buf) {  
    return *((uint16_t*)buf);  
}  
  
uint32_t get_u32(void* buf) {  
    return *((uint32_t*)buf);  
}  
  
uint64_t get_u64(void* buf) {  
    return *((uint64_t*)buf);  
}
```



```
get_u8:  
    movzx    eax, BYTE PTR [rdi]  
    ret  
  
get_u16:  
    movzx    eax, WORD PTR [rdi]  
    ret  
  
get_u32:  
    mov      eax, DWORD PTR [rdi]  
    ret  
  
get_u64:  
    mov      rax, QWORD PTR [rdi]  
    ret
```

- GCC output available [here](#)

Indexed Addressing Example

```
int a[20];  
...  
int i, sum=0;  
for (i=0; i<20; i++)  
    sum += a[i];
```

```
        xor     rax, rax  
        xor     rbx, rbx  
        lea     rcx, a  
sumloop:  
        mov     rdx, [rcx + rax * 4]  
        add     rbx, rdx  
        inc     rax  
        cmpl    rax, 19  
        jle     sumloop
```

- GCC version available [here](#)

Memory endianness

Data on most modern systems is stored *backwards*, in *little endian*.

```
mov eax, 0xc001ca75 # sets rax to  
mov rcx, 0x10000  
mov [rcx], eax      # stores data as  
mov bh, [rcx]      # reads 0x75
```

c0	01	ca ^{ah}	75 ^{al}
75 ^{0x10000}	ca ^{0x10001}	01 ^{0x10002}	c0 ^{0x10003}

Bytes are *only* shuffled for multi-byte stores and loads of registers to memory!
Individual bytes *never* have their bits shuffled.

Yes, writes to the stack behave just like any other write to memory.

Why little endian?

Intel created the 8008 for a company called Datapoint in 1972.

Datapoint used little endian for easier implementation of *carry* in arithmetic!

Intel used little endian in 8008 for compatibility with Datapoint's processes!

Every step in the evolution between 8008 and modern x86 maintained some level of binary compatibility with its predecessor.

<https://en.wikipedia.org/wiki/Endianness>



Memory endianness – Let's verify it with GDB

```
.intel_syntax noprefix
```

```
.global _start  
.section .text
```

```
_start:
```

```
sub rsp, 8  
mov rax, 0x8899aabbccddeeff  
mov [rsp], rax  
add rsp, 8
```

```
# Exiting  
mov rax, 60  
xor rdi, rdi  
syscall
```

Not
necessary



1 Write the program

```
$ gcc -static -nostdlib -o endianness endianness.s  
$ gdb ./endianness
```

2 Compile & debug

```
(gdb) disass _start
```

Dump of assembler code for function `_start`:

```
0x000000000000401000 <+0>:      sub    rsp, 0x8  
0x000000000000401004 <+4>:      movabs  rax, 0x8899aabbccddeeff  
0x00000000000040100e <+14>:     mov     QWORD PTR [rsp], rax  
0x000000000000401012 <+18>:     add     rsp, 0x8  
0x000000000000401016 <+22>:     mov     rax, 0x3c  
0x00000000000040101d <+29>:     xor     rdi, rdi  
0x000000000000401020 <+32>:     syscall
```

3

Obtain the correct
instruction offset

Memory endianness – Let's verify it with GDB

4 Set the breakpoint and run

```
(gdb) b *(_start + 18)
Breakpoint 1 at 0x401012
(gdb) r
Starting program: /home/user/SP/Lab2/endianness

Breakpoint 1, 0x0000000000401012 in _start ()
```

5 Verify

```
(gdb) i r rax
rax                0x8899aabbccddeeff  -8603657889541918977
(gdb) x/8xb $rsp
0x7fffffffdd58: 0xff    0xee    0xdd    0xcc    0xbb    0xaa    0x99    0x88
```



Assembly instructions



Arithmetic Instructions

- Simple instructions

- `add dest, source` $\text{dest} = \text{source} + \text{dest}$
- `sub dest, source` $\text{dest} = \text{dest} - \text{source}$
- `inc dest` $\text{dest} = \text{dest} + 1$
- `dec dest` $\text{dest} = \text{dest} - 1$
- `neg dest` $\text{dest} = \sim \text{dest} + 1$
- `cmp source1, source2` $\text{source2} - \text{source1}$

- Multiply
 - `mul` (unsigned) or `imul` (signed)
 - Performs signed multiplication and stores the result in the second operand. If the second operand is left out, it is assumed to be **rax**. The full result is stored in the double-word **rdx:rax**
- Divide
 - `div` (unsigned) or `idiv` (signed)
 - Divides the contents of the double-word contained in the combined **edx:eax** registers by the value in the register or memory location specified. The **eax** register contains the resulting quotient, and the **edx** register contains the resulting remainder

Bitwise logic instructions

- Simple instructions

- | | | |
|-------|---------------|-----------------------|
| • and | dest, source, | dest = source & dest |
| • or | dest, source, | dest = source dest |
| • xor | dest, source, | dest = source ^ dest |
| • not | dest | dest = ~dest |
| • sal | dest, source, | dest = dest << source |
| • sar | dest, source, | dest = dest >> source |

Control Flow

- We obtain control flow using two instructions:

```
cmp rax,0
```

```
je end_loop
```

- The first one is the **cmp** instruction which compares two values, and stores the result of the comparison in the status register RFLAGS
- The second operand is subtracted from the first one and then the status flags is set in the same manner as the SUB instruction. When an immediate value is used as an operand, it is sign-extended to the length of the first operand.
- **je** is the flow control instruction JUMP which says to jump to the label end_loop depending on the values stored in the status register and on the condition expressed



Types of Jumps

- **je**: Jump if the values were equal
- **jl**: Jump if the destination was greater than the source
- **jge**: Jump if the destination was greater than or equal to the source
- **jl**: Jump if the destination was less than the source
- **jle**: Jump if the destination is less than or equal to the source
- **jmp**: Jump no matter what. This does not need to be preceded by a comparison

Calling Convention / Application Binary Interface

- Internally, an assembly program may follow no convention
 - To put it simply, almost any register can be used arbitrarily to perform operations.
- When interacting with the environment (e.g the stack), external libraries or syscalls, every program must adhere to what is considered a calling convention:
 - Specific registers must be used to pass parameters
 - Specific registers must be saved by a called function and restored upon return
 - ...

Calling Convention / Application Binary Interface

Register	Usage	Preserved across function calls
%rax	temporary register; with variable arguments passes information about the number of vector registers used; 1 st return register	No
%rbx	callee-saved register	Yes
%rcx	used to pass 4 th integer argument to functions	No
%rdx	used to pass 3 rd argument to functions; 2 nd return register	No
%rsp	stack pointer	Yes
%rbp	callee-saved register; optionally used as frame pointer	Yes
%rsi	used to pass 2 nd argument to functions	No
%rdi	used to pass 1 st argument to functions	No
%r8	used to pass 5 th argument to functions	No
%r9	used to pass 6 th argument to functions	No
%r10	temporary register, used for passing a function's static chain pointer	No
%r11	temporary register	No
%r12-r14	callee-saved registers	Yes
%r15	callee-saved register; optionally used as GOT base pointer	Yes

Calling Convention for kernel

1. User-level applications use as integer registers for passing the sequence `%rdi, %rsi, %rdx, %rcx, %r8` and `%r9`. The kernel interface uses `%rdi, %rsi, %rdx, %r10, %r8` and `%r9`.
2. A system-call is done via the `syscall` instruction. The kernel destroys registers `%rcx` and `%r11`.
3. The number of the syscall has to be passed in register `%rax`.
4. System-calls are limited to six arguments, no argument is passed directly on the stack.
5. Returning from the `syscall`, register `%rax` contains the result of the system-call. A value in the range between -4095 and -1 indicates an error, it is `-errno`.

Calling Convention Example – Parameters

```
int __attribute__((noinline)) super_sum(int a, int b, int c, int d, int e, int f, int g)
{
    return a + b + c + d + e + f + g;
}
```

rdi, rsi, rdx, rcx, r8, r9, stack

super_sum:

add edi, esi a = a + b
add edx, edi c = c + b
add ecx, edx d = d + c
add r8d, ecx e = e + d
lea eax, [r8+r9] res = e + f
add eax, DWORD PTR [rsp+8] res = res + g
ret

Full example [here](#)

- Assembly programs use system calls for performing more complicated operations
- Linux system calls are called in the following way:
 - Store the system call number in rax
 - Set up the arguments to the system call in rdi, rsi, rdx, r10, r8, r9
 - Exec the syscall instruction
 - The result is usually returned in EAX

Syscall table

%rax	System call	%rdi	%rsi	%rdx	%r10	%r8	%r9
0	sys_read	unsigned int fd	char *buf	size_t count			
1	sys_write	unsigned int fd	const char *buf	size_t count			
2	sys_open	const char *filename	int flags	int mode			
3	sys_close	unsigned int fd					
4	sys_stat	const char *filename	struct stat *statbuf				
5	sys_fstat	unsigned int fd	struct stat *statbuf				

Information on the order of registers can be found on page 124 of the x86_64 ABI paper at <http://www.x86-64.org/documentation/abi.pdf>

Example

Remember system calls? It's an instruction that makes a *call* into the Operating System.

`syscall` triggers the system call specified by the value in `rax`.

arguments in `rdi`, `rsi`, `rdx`, `r10`, `r8`, and `r9`

return value in `rax`

Reading 100 bytes from stdin to the stack:

```
n = read(0, buf, 100);
```

```
mov rdi, 0 # the stdin file descriptor
```

```
mov rsi, rsp # read the data onto the stack
```

```
mov rdx, 100 # the number of bytes to read
```

```
mov rax, 0 # system call number of read()
```

```
syscall # do the system call
```

`read` returns the number of bytes read via `rax`, so we can easily **write** them out:

```
write(1, buf, n);
```

```
mov rdi, 1 # the stdout file descriptor
```

```
mov rsi, rsp # write the data from the stack
```

```
mov rdx, rax # the number of bytes to write (same as what we read in)
```

```
mov rax, 1 # system call number of write()
```

```
syscall # do the system call
```

Reserving space

variables:

`.byte 0xff`

`.word 0xaabb`

`.int 0xccddeeff`

`.quad 0x1122334455667788`

`.space 10, 0xcc`

`.string "Ciao"`

- Specific assembler directives to reserve space

`~$ hd embryoasm1.tex`

00000000	48	c7	c7	37	13	00	00	66	c7	04	25	67	45	23	01	00	H..7...f..%gE#..
00000010	00	8a	07	ff	bb	aa	ff	ee	dd	cc	88	77	66	55	44	33	wfUD3
00000020	22	11	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	".....Ciao
00000030	00																.



Writing some code

Hello world

```
.intel_syntax noprefix
```

```
.global _start
```

```
.section .text
```

```
_start:
```

```
mov rax, 1      # write(  
mov rdi, 1      #   STDOUT_FILENO,  
lea rsi, message #   "Hello, world!\n",  
mov rdx, 13     #   sizeof("Hello, world!\n")  
syscall         # );
```

```
mov rax, 60     # exit(  
mov rdi, 0      #   EXIT_SUCCESS  
syscall         # );
```

```
message:
```

```
.ascii "Hello, world\n"
```

Tells the assembler that we are using INTEL's syntax

`_start` symbol will be exported: linker/loader will see it, and code will be executed from there

Defines the begin of `.text` section, an executable code region

Defines `_start` symbol

Performs a syscall

Compile an Assembly program

`gcc -nostdlib -static -o output_elf input_file`

- *-nostdlib* tells the linker to not use the standard system startup files or libraries:
 - The program will start exactly from our defined `_start` symbol
- *-static* disables shared-libraries linking and PIE
 - The program will start exactly from our defined `_start` symbol

PwnCollege assembly

```
hacker@embryoasm_level1:~$ /challenge/embryoasm_level1
```

```
Welcome to EmbryoASMLevel1
```

```
=====
```

To interact with any level you will send raw bytes over stdin to this program.
To efficiently solve these problems, first run it once to see what you need
then craft, assemble, and pipe your bytes to this program.

In this level you will work with registers_use! Please set the following:
* rdi = 0x1337

Please give me your assembly in bytes (up to 0x1000 bytes):

Executing your code...

----- CODE -----

ERROR: Invalid memory read (UC_ERR_READ_UNMAPPED)

Sorry, no flag :(.

Run the challenge to
understand the requirements

PwnCollege assembly

```
embryoasm1.s
```

```
.intel_syntax noprefix
```

```
.global _start
```

```
.section .text
```

```
_start:
```

```
    mov rdi, 0x1337
```

- Solve the challenge
- Compile, extract only the relevant code, pass the code bytes to the challenge

```
hacker@embryoasm_level1:~$ gcc -static -nostdlib -o embryoasm1 embryoasm1.s
```

```
hacker@embryoasm_level1:~$ objcopy --dump-section .text=embryoasm1.tex embryoasm1
```

```
hacker@embryoasm_level1:~$ cat embryoasm1.tex | /challenge/embryoasm_level1
```